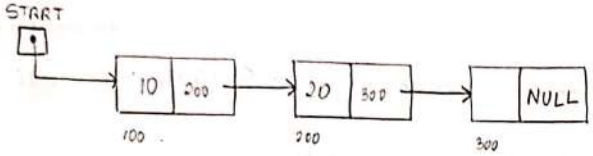
Linked List

→ when the member of a structure points to the str. itself it is known as SELF REFERENTIAL STRUCTURE.

Eg: linked list, graph, tree.

STRUCTURE :

```
Struct node {
        int info;
        Struct node *link;
} Struct node * first;
```



→ We cannot directly access the structure, they need to be accessed by structure variable.

DISPLAY / TRAVERSAL :

Time complexity : O(n)

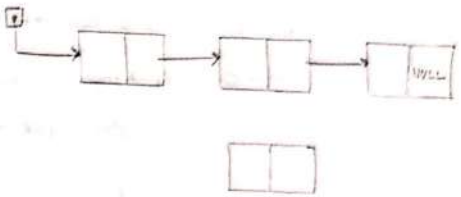
```
Void T_SLL(){
        Struct node * ptr;
        ptr = first;
        while (ptr != NULL){
                printf ("%d", ptr → info);
                ptr = ptr → link;
                }
}
```

INSERTION : (Beginning)

Time complexity : O(1)

```
void insert_beg(){
        Struct node * new;
        new = (Struct node *) malloc (sizeof (Struct node));
        Scanf ("%d", & new → info);
        new → link = first;
        first = new;
}
```

→ an operator not f^n



INSERTION : (End)

Time complexity : O(n)

```
void insert_end(){
        Struct node * new, * ptr;
        new = (Struct node *) malloc (sizeof (Struct node));
        Scanf ("%d", & new → info);
        while (ptr → link != NULL){
                ptr = ptr → link;
                }
        new → link = NULL;
        ptr → link = new;
}
```

Scanned with OKEN Scanner

INSERTION : (Middle) ; after a given value.

```
void ins_mid () {

    Struct node * new, * ptr;
    int value;
    new = (struct node *) malloc (sizeof (struct node));
    scanf ("%d", & value);
    scanf ("%d", & new -> info);
    while ( ptr -> info ! = value ) {
            ptr = ptr -> link;
            }
    new -> link = ptr -> link;
    ptr -> link = new;
}
```

DELETION : (Beginning)

Time complexity : O(1)

```
void del_beg ( ) {
    Struct node * ptr;
    ptr = first ;
    first = first -> link;
    free (ptr);
}
```

DELETION : (End)

```
void del_end ( ) {
    Struct node *ptr, * temp ;
    ptr = first;
    while (ptr->link = NULL) {
            temp = ptr;
            ptr = ptr -> link;
            }
    temp -> link = NULL ;
    free (ptr) ;
}
```

DELETION : (Middle) ∴ given data.

```
void del_mid ( ) {
    Struct node *ptr, * temp ; int data;
    ptr = first ;
    scanf ("%d", &data) ;
    while ( ptr -> info ! = data) {
            temp = ptr;
            ptr = ptr -> link ;
            }
    temp -> link = ptr -> link ;
    free (ptr);
}
```

Scanned with OKEN Scanner

CREATION of SINGLE LINKED LIST:

```
struct node {
        int info;
        struct node * link;
        }*head;
void main() {
    struct node *ptr, cpt; char ch;
    int n, data;
  ptr = (struct node *) malloc (size of (struct node));
  scanf ("%d", ptr->info);
  first = ptr;
  ptr -> link = NULL;
do {
    cpt = (struct node *) malloc (sizeof (structnode));
    scanf ("%d", cpt -> info);
    ptr -> link = cpt;
    ptr = cpt;
    printf (" Y for more node, N for exit ");
    ch = getch (ch);
  } while (ch == "Y");
  ptr -> link = NULL;
}
```

— Circular linked List —

CREATION :

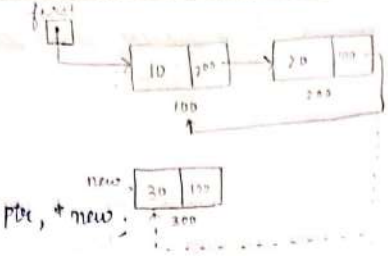
→ In circular linked list, the link of last node points to the first node instead of NULL.

```
void create () {
        struct node *ptr, *cpt;
        Char ch;
        ptr = (struct node*) malloc (size of (struct node));
        scanf ("%d", &ptr -> info);
        first = ptr;
        ptr -> link = ptr;
        do {
            cpt = (struct node*) malloc (size of (struct node));
            scanf ("%d", &cpt -> info);
            ptr -> link = cpt;
            cpt = ptr;
            printf (" Y for new node, N for exit ");
            ch = getch();
            } while (ch = "Y");
            ptr -> link = first;
            }
```

Scanned with OKEN Scanner

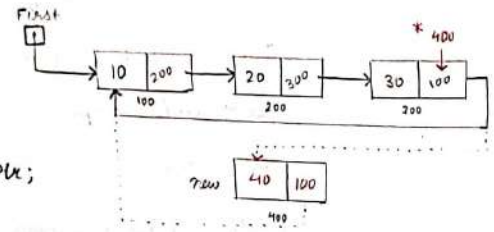
INSERTION :

i) Beginning




```
void insert_beg() {
        struct node *ptr, *new;
        ptr = first;
        new = (struct node *) malloc ( sizeof (struct node));
        scanf ("%d", & new -> info);
        new -> link = first;
        first = new;
        while ( ptr -> link != first -> link) {
                ptr = ptr -> link;
                ptr -> link = first;
                }
}
```

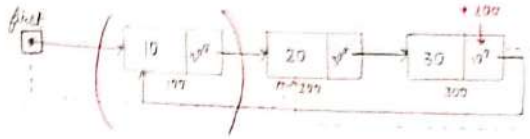
(ii) End




```
void insert_end() {
        struct node *new, *ptr;
        ptr = first;
        new = (struct node *) malloc (sizeof (struct node));
        scanf ("%d", & new -> info);
        while ( ptr -> link != first ) {
                ptr = ptr -> link;
                new -> link = first;
                ptr -> link = new;
                }
}
```

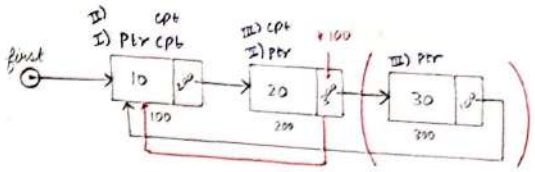
DELETION :

i) Beginning




```
void del_beg() {
        struct node *ptr, *cpt;
        cpt = first;
        ptr = first;
        while ( cpt -> link != first) {
                cpt = cpt -> link;
                }
        cpt -> link = first -> link;
        free (ptr);
}
```

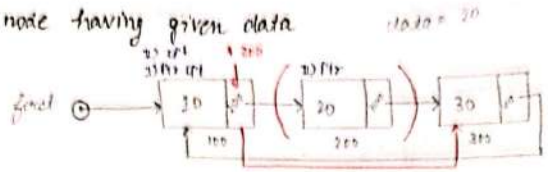
ii) End




```
void del_end() {
        struct node *ptr, *cpt;
        ptr = cpt = first;
        while ( ptr -> link != first ) {
                cpt = ptr;
                ptr = ptr -> link;
                }
        cpt -> link = first;
        free (ptr);
}
```

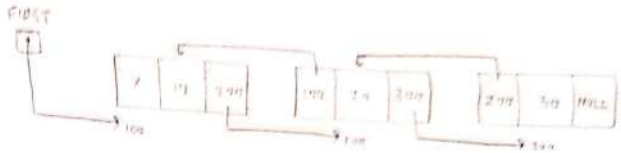
Scanned with OKEN Scanner

iii) Middle : remove node having given data




```
void del_mid(){

    struct node *ptr, *cpt;
    int data;
    scanf("%d", &data);
    ptr = cpt = first;

    while( ptr->info != data){
            cpt = ptr;
            ptr = ptr->link;
            }

  cpt -> link = ptr->link;
  free(ptr);
  }
```

— Doubly Linked list —



Structure:

```
struct node {
        int info;
        struct node * lpt;
        struct node * rpt;
        }
```

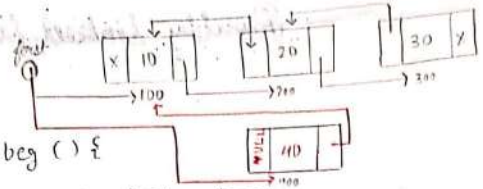
CREATION :

```
void create () {
        struct node *ptr, *cpt,
        ptr = (struct node *) malloc (sizeof (struct node));
        ptr -> lpt = ptr -> rpt = NULL;
        scanf ("%d", & ptr -> info );
        first = ptr;

        while do {
                cpt = (struct node *) malloc ((struct node) sizeof ));
                scanf ("%d", & cpt -> info );
                ptr -> rpt = cpt ;
                cpt -> lpt = ptr ;
                ptr = cpt;
                printf (" Y for add node or N for exit");
                scanf ("%c", & chara);    // getch(ch);
                } while ( ch = 'y');

        ptr -> rpt = NULL;
        }
```

Scanned with OKEN Scanner

INSERTION :

(i) Beginning :



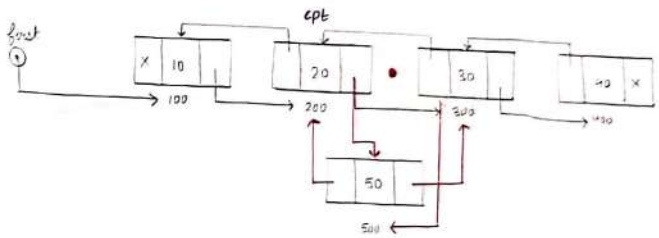

```
void insert_beg () {
    struct node *ptr, *cpt;
    ptr = (struct node*) malloc (sizeof (struct node));
    Scanf ("%d", ptr->info);
    ptr->lpt = NULL;
    ptr->rpt = first;
    first->lpt = ptr;
    first = ptr;
}
```

(ii) End :

```
void insert_end () {
    struct node *ptr, cpt;
    ptr = (struct node *) malloc (sizeof (struct node));
    Scanf ("%d", &ptr->info);
    cpt = first;
    while ( cpt->lpt != NULL) {
            cpt = cpt->rpt ;

    ptr->lpt = cpt;
    ptr->rpt = NULL;
    cpt->rpt = ptr;

}
```

(iii) Middle : After given data

```
void insert_mid () {
    struct node *ptr, *cpt;
    int data;
    scanf ("%d", &data);
    ptr = (struct node*) malloc (sizeof (struct node));
    scanf ("%d", &ptr->info);
    cpt = first;
    while (cpt->info != data) {
            cpt = cpt->rpt;
    }
    ptr->lpt = cpt;
    ptr->rpt = cpt->rpt;
    cpt->rpt->lpt = ptr;   imp step
    cpt->rpt = ptr;
}
```



data = 20

Scanned with OKEN Scanner

DELETION:

(i) Beginning:

```
void del_beg () {
        Struct node * ptr;
        ptr = first;
        ~~fptr~~ → rpt → lpt = NULL;   // use first instead of
        first = ~~rpt~~ first → rpt;   ptr to not
                                       distord the list.
        free ( ptr);
}
```

(ii) End:

```
void del_end () {
        Struct node * ptr * cpt;
        ptr = first;
        cpt = first;
        while ( ptr → rpt ! = NULL ) {
                cpt = ptr
                ptr = ptr → rpt;
                }
        cpt → rpt = NULL;
        free (ptr);
}
```

(iii) Middle:

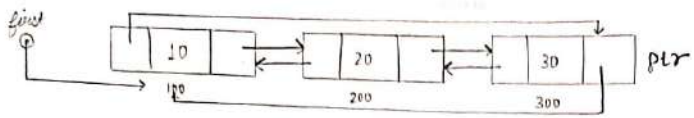
```
void del_mid () {
        Struct node * ptr, * cpt;
        ptr = first;
        int data;
        scanf ("%d", &data);
        while ( ptr → info ! = data ) {
                ptr = ptr → rpt;
                }
        ptr → lpt → rpt = ~~rpt~~ ptr → rpt;
        free (ptr);
```

BACKWARD TRAVERSING OF DOUBLY LL:

```
void backtrack () {
        Struct node * ptr;
        ptr = first;
        while ( ptr → rpt ! = NULL ) {
                ptr = ptr → rpt;
                }
        while ( ptr → lpt ! = NULL ) { // (ptr ! = first)
                printf ("%d", ptr → info);
                ptr = ptr → lpt;
                }
        printf("%d", ptr → info);
}
```

Scanned with OKEN Scanner

Doubly Circular Linked List



Structure :

```
struct node {
        int info;
        struct node *lpt;
        struct node *rpt;
} *first;
```

CREATION :

```
void create () {
        struct node * ptr ; new; char ch;
        ptr = (structnode*) malloc ( sizeof ( structnode *) );
        scanf ("%d", & ptr → info);
        ptr → lpt = ptr → rpt = ptr;
        first = ptr;
        do {
            new = (struct node*) malloc (sizeof ( struct node));
            scanf ("%d", & new → info);
            ptr → rpt = new;
            new → lpt = ptr;
            ptr = new;
            printf (" Y for new node, N for ex't ");
            getch (ch);
        } while (ch == "Y");
        ptr → rpt = first;
        first → lpt = ptr;
}
```

DISPLAY :

i) Forward

```
void for_display () {
        struct node * ptr;
        ptr = first;
        while ( ptr → rpt ! = first ) {
                printf ("%d", ptr → info);
                ptr = ptr → lpt;
                }
        printf ("%d", ptr → info);   // ptr will be on last node
}
```

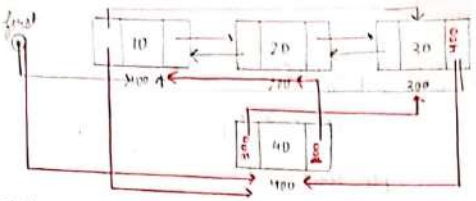
ii) Backward

```
void back_display ( ) {
        struct node * ptr ;
        ptr = first ;
        while ( ptr → rpt ! = first ) {
                ptr = ptr → rpt ;
                }
        while ( ptr ! = first ) {
                printf ("%d", ptr → info) ;
                ptr = ptr → lpt;
                }
        printf ("%d", ptr → info);
}
```

Scanned with OKEN Scanner

INSERTION :

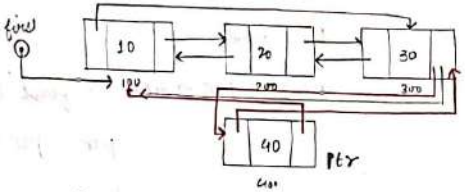
i) Beginning




```
void insert_beg(){
    struct node * ptr, * cpt ;
    ptr = (struct node *) malloc (sizeof (structnode)) ;
    scanf ("%d", & ptr->info);

    ptr -> rpt = first;
    ptr -> lpt = first -> lpt;
    first = ptr;
    ptr -> rpt -> lpt -> rpt = ptr ;
    ptr -> rpt -> lpt = ptr;
}
```

ii) Middle : after given data (Same as Doubly linked list)

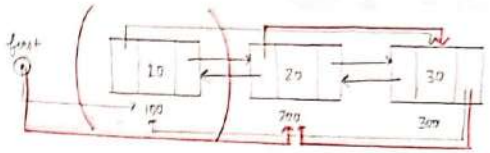
iii) End




```
void insert_end(){
    struct node * ptr ;
    ptr = (struct node *) malloc (sizeof (structnode));
    scanf ("%d", & ptr -> info) ;
    ptr -> rpt = first ;
    ptr -> lpt = first -> lpt ;
    first -> lpt -> rpt = ptr;
    first -> lpt = ptr ;
```

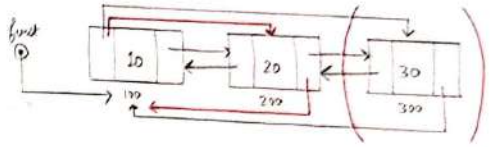
DELETION :

i) Beginning




```
void del_beg(){
    struct node * ptr;
    ptr = first ;
    first = first -> rpt ;
    first -> lpt = ptr -> lpt;
    first -> lpt -> rpt = first ;
    free (ptr);
}
```

ii) End




```
void del_end(){
    struct node * ptr;
    ptr = first -> lpt;
    first -> lpt = ptr -> lpt;
    ptr -> lpt -> rpt = first ;
    free (ptr);
}
```

iii) Middle Same as doubly linked list

Scanned with OKEN Scanner

Hashing

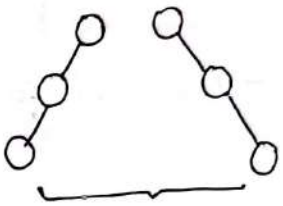
→ 2^{20} elements operations

① Linear search : $O(n)$

② Binary Search : $O(\log n)$ [Sorted array] needed

③ BST: $O(h)$

h, height → $O(\log n)$ [20 comparisons]



drawbacks

④ AVL tree: (Strictly balanced)

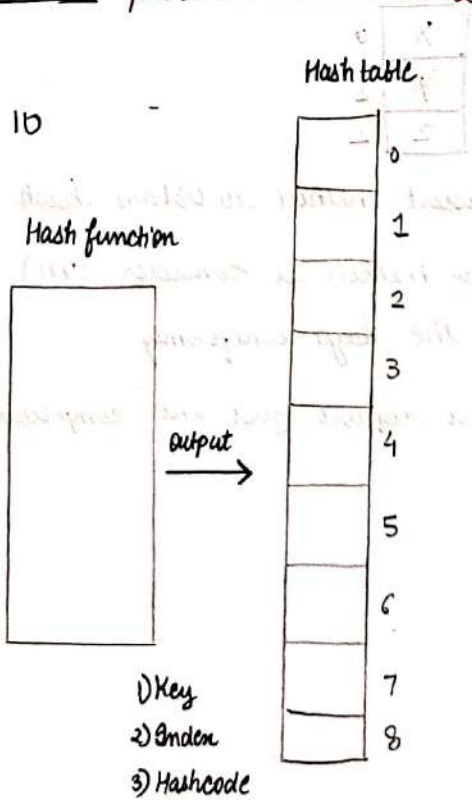
⑤ Red Black tree : For frequent insertion / deletion / searching.

⑥ Splay tree : For searching the recently accessed elements

⑦ HASHING : $\boxed{O(1)}$ provided there is no collission.

[10, 15, 17, 20] $m = 10$

$H(K) = K \% m$



Division
Mid square
folding
multiplication

Scanned with OKEN Scanner

Properties of a good hash function:

→ It should be easy to compute.

→ After applying hash f^n the keys should be uniformly distributed else it will cause COLLISSION.

→ Hash function cannot remove collission 100% but can try maximum reduction of collission.

→ Few hash functions are:

(i) Division method (most commonly used)
(ii) Mid-square method
(iii) Folding / chopping method
(iv) Multiplication method

① Division method:

Size of hash table (m) = 5

Key value pair = { 10: "X", 11: "Y", 12: "Z" }

$\boxed{\text{Hash } f^n: H(k) = k \% m}$

$H(10) = 10 \% 5 = 0 \longrightarrow$ X | 0
$H(11) = 11 \% 5 = 1 \longrightarrow$ Y | 1
$H(12) = 12 \% 5 = 2 \longrightarrow$ Z | 2

→ It is the simplest and easiest method to obtain hash value.

→ The best practise using this method is consider (m) as the prime no., So that we can distribute the keys uniformly.

→ It is also very fast as it requires just one computation and that is modulus (%).

② Mid-Square method:

→ It's a 2 step process of computing the hash value.

→ Given a key : value pair the hash f^n would be calculated by

i) Square the key
ii) Chose some digit from middle of the no. and obtain the hash value.

Eg: { 50: "X", 45: "Y", 79: "Z" } size of table : 1000

array : 0 — 999 (3 digits)

1) 50 → $\boxed{2500}$: H(500) = X
2) 45 → $\boxed{2250}$: H(250) = Y

Y	250
X	500
⋮	

→ All the digits in the key are utilized to ~~continue~~ contribute to the index thus increasing the performance of DS.

→ Collission might occur but we can try to reduce or handle them.

→ The disadvantage is when we square a huge no. it will cross the range of integer and OVERFLOW may occur.

③ Folding / chopping method:

$H(K) = (\text{size of equal size segments}) \% (\text{size of hash table})$

K: 10 digit no. Size: 100 (0-99)

$H(K) = (K_1K_2 + K_3K_4 + K_5K_6 + K_7K_8 + K_9K_{10}) \% m$

Eg: { 1234: "X", 5678: "Y" } m = 100

$H(1234) = 12 + 34 = 46 \% 100 = 46$ → "X"

$H(5678) = 56 + 78 = 134 \% 100 = 34$ → "Y"

Scanned with OKEN Scanner

④ Multiplication method:

→ We must chose a constant b/w 0-1 say (A).

→ Multiply the key with (A).

→ Take only the decimal / fractional part from product & multiply it by table size.

→ The hash will be the floor of the above result.

$$\boxed{H(x) = \text{floor}\,(\text{size} * (\text{Key} * A))}$$

Eg: 1234 → X A = 0.56

5678 → Y size, m = 100.

$H(1234) = (1234) * (0.56) = 691.04$

→ $(0.04)(m) \rightarrow 4 \rightarrow \text{floor}(4) = 4 \longrightarrow X$

$H(5678) = (5678) * (0.56) = 3179.68$

$m = 100 \rightarrow (0.68)(m) \rightarrow 68 \rightarrow \text{floor}(68) = 68 \longrightarrow Y$

$m = 99 \rightarrow (0.68)(99) \rightarrow 67.32 \rightarrow \text{floor}(67.32) = 67 \longrightarrow Y$

Collision removal techniques

- Open hashing (closed addressing) → (Chaining)
- Closed hashing (open addressing)
  - [Linear probing]
  - [Quadratic probing]
  - [Double hashing]

Q) A = 3, 2, 9, 6, 11, 13, 7, 12

$H(K) = 2K+3$

$m = 10$

use division method: $h(K) \cdot 1 \cdot m$

& closed addressing

$H(3) = 6+3 = 9$; $9 \cdot 1 \cdot 10 = 9$

$H(2) = 4+3 = 7$; $7 \cdot 1 \cdot 10 = 7$

$H(9) = 18+3 = 21$; $21 \cdot 1 \cdot 10 = 1$

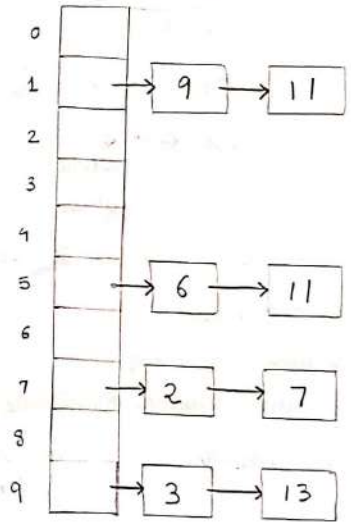
$H(6) = 12+3 = 15$; $15 \cdot 1 \cdot 10 = 5$

$H(11) = 22+3 = 25$; $25 \cdot 1 \cdot 10 = 5$

$H(13) = 26+3 = 29$; $29 \cdot 1 \cdot 10 = 9$

$H(7) = 14+3 = 17$; $17 \cdot 1 \cdot 10 = 7$

$H(12) = 28+3 = 31$; $31 \cdot 1 \cdot 10 = 1$



→ The worst case in closed addressing is when all keys are at same index.

→ On an avg the size of LL is given by (n/m)

Scanned with OKEN Scanner

Load factor : $\left(\frac{n}{m}\right)$ → represented by (lambda λ)

for seperate chaining : $\boxed{1 \leq \lambda \leq \infty}$

for,
- Linear
- quadratic
- double

$$\begin{cases} h = 5 \\ \lambda \leq 1 \\ \lambda = 3/4 = 75\% \\ \lambda = 1 = 100\% \\ \lambda = 0 \text{ (empty)} \end{cases}$$

Closed hashing

① **Linear probing** :

$$\boxed{H(k,i) = \{H(k) + i\} \% m}$$

hash f^n

keys

probe no/ coll^n no.

(0 to m-1)

size of table

keys : 10 20 30 40 50 60 70 80 $(m=10)$ $(H(k) = k \% 10)$

1) No. of probes
2) Probing sequence of 80
3) Order of elements in hash table

10: $\{0+0\} \% 10 = 0$

20: $\{0+0\} \% 10 = 0$
$\{0+1\} \% 10 = 1$

key	location	probe
10	0	1
20	(0) 1	2
30	(0)(1)(2) 3	3
40	(0 1 2 3) 4	4
50	(0 1 2 3 4) 5	5
60	(0 1 2 3 4 5) 6	6
70	(0 1 2 3 4 5 6) 7	7
80	(0 1 2 3 4 5 6 7) 8	8

10	0
20	1
30	2
40	3
50	4
60	5
70	6
80	7
	8
	9

→ Linear probing suffers from the problem of PRIMARY CLUSTERING.

→ Greater the size of cluster the probability that cluster will grow is very high.

→ The probing sequence generated by linear probing is (m) and keys which are mapped to the same location will follow the same probing sequence.

Scanned with OKEN Scanner

2) Quadratic probing :

→ keys mapped to the same location will follow same col^n resolution path or same probing sequence because of which keys will not beable to utilize free space.

$$\boxed{H(K,i) = \{H(K) + C_1 * i + C_2 * i^2\} \cdot \% \cdot m}$$

keys: 10 20 30 40 50 60 70 80

$C_1 = 0$
$C_2 = 1$
$m = 10$
$H(K) = K \cdot \% \cdot 10$

$H(K,i) = \{H(K) + i^2\} \cdot \% \cdot m$

key	location	probes
10	0	1
20	0 1	2
30	0 1 4	3
40	0 1 4 9	4
50	0 1 4 9 6	5
60	0 1 4 9 6 5	6

⇒

10	0
20	1
	2
	3
30	4
60	5
50	6
	7
	8
40	9

10: $\{0+0\} \cdot \% \cdot 10 = 0$

20: $\{0+0\} \cdot \% \cdot 10 = 0$; $\{0+1\} \cdot \% \cdot 10 = 1$

30: $\{0+0\} \cdot \% \cdot 10 = 0$; $\{0,1\} = 1$; $\{0,2\} = \{0+2\} \cdot \% \cdot 10 = 4$

40: $\{0,0\} = 0$; $\{0,1\} = 1$; $\{0,2\} = 4$; $\{0,3\} = \{0+9\} \cdot \% \cdot 10 = 9$

50: $\{50,0\} = 0$; $\{50,1\} = 1$; $\{50,2\} = 4$; $\{50,3\} = 9$; $\{50,4\} = \{0+16\} \cdot \% \cdot 10 = 6$

60: $\{60,0\} = 0$; $\{60,1\} = 1$; $\{60,2\} = 4$; $\{60,3\} = 9$; $\{60,4\} = 6$; $\{60,5\} = \{0+25\} \cdot \% \cdot 10 = 5$

70: $\{70,0$ . . . . . $\{70,6\} = \{0+36\} \cdot \% \cdot 100 = 6$

3) Double hashing :

$$\boxed{H(K,i) = \{H_1(K) + H_2(K) * i\} \cdot \% \cdot m}$$

8) keys: 13, 26, 39, 52

$H_1(K) = K \cdot \% \cdot 13$ $\quad$ $H_2(K) = K \cdot \% \cdot 12$

Quadratic

13: $\{13,0\} = 0$

26: $\{26,0\} = 0$; $\{26,1\} = 1$

39: $\{39,0\} = 0$; $\{39,1\} = 1$; $\{39,2\} = 4$

54: $\{52,0\}$ . . . . . $\{52,3\} = 9$

Sequence: 13 26 39 54

index : 0 1 4 9

Double hashing

13: $\{13,0\} \rightarrow \{0 + 1*0\} \cdot \% \cdot 10 = 0$

26: $\{26,0\} \rightarrow \{0 + 2*0\} \cdot \% \cdot 0 = 0$ ✗

$\{26,1\} \rightarrow \{0 + 2*1\} \cdot \% \cdot 0 = 2$

39: $\{39,0\} \rightarrow \{0 + 3*0\} \cdot \% \cdot 10 = 0$

$\{39,1\} \rightarrow \{0 + 3*1\} \cdot \% \cdot 10 = 3$

52: $\{52,0\} = \{0 + 4*0\} \cdot \% \cdot 0 = 0$

$\{52,1\} = \{0 + 4*1\} \cdot \% \cdot 10 = 4$

Sequence: 13 26 39 52

index : 0 2 3 4

→ No. of probing sequence by double hashing is (m^2).

Probing sequence comb^n = Seq $H_1(K)$ + Seq $H_2(K)$

$\underbrace{m \qquad m}_{(m^2)}$

Scanned with OKEN Scanner

Rehashing:

Extendible hashing / Dynamic hashing:

keys: 16, 4, 6, 22, 24, 10, 31, 7, 9, 20, 26

Bucket size: 3

Hashing f^n: Suppose the global depth is (X) then the hash f^n makes X LSB.

Step1) Initialize the bucket depth and global depth of directory.

Step2) Convert data into a binary representation.

Step3) Consider the global depth the no. of LSB in data.

Step4) Map the data according to the i'd of a directory.

Step5) Check the following condn if overflow occurs.

i) Global depth = bucket depth then split the bucket in 2 parts and increment the GD and BD by 1, Rehash the elements of split buckets.

ii) When gD is > BD, split only the bucket and increment the BD only and then, Rehash the elements of split buckets.

Step6) Repeat the steps above for each elements.

Key terms used in extendible hashing:

1) Directory: It stores the addresses of buckets in pointers.
I'D is assigned to each directory which may change each time when directory expansion takes place

2) Buckets: These are used to hash the actual data.

3) Global depth: It is associated with directory, it denotes the no. of bits which are used by hash f^n to categorize the keys.
Global depth is equal to no. of bits in directory I'D.

Scanned with OKEN Scanner

Local depth: It is associated with bucket's local depth in accordance with global depth decide the action to be performed, in case an overflow occurs LD is always < = global depth.

advantages:

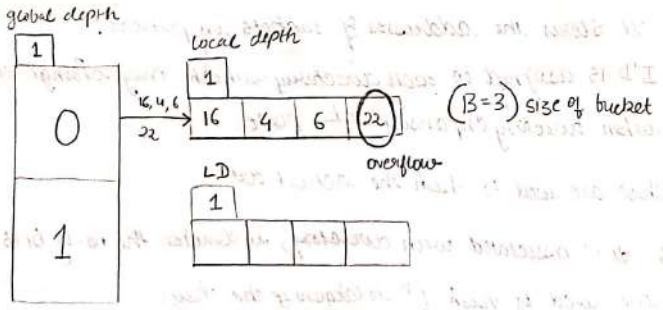
1) It is the best tech. to accomodate the grp or shrinkage of database.
2) Data retrieval is less expensive.
3) No problem of data loss since the storage capacity increases dynamically.

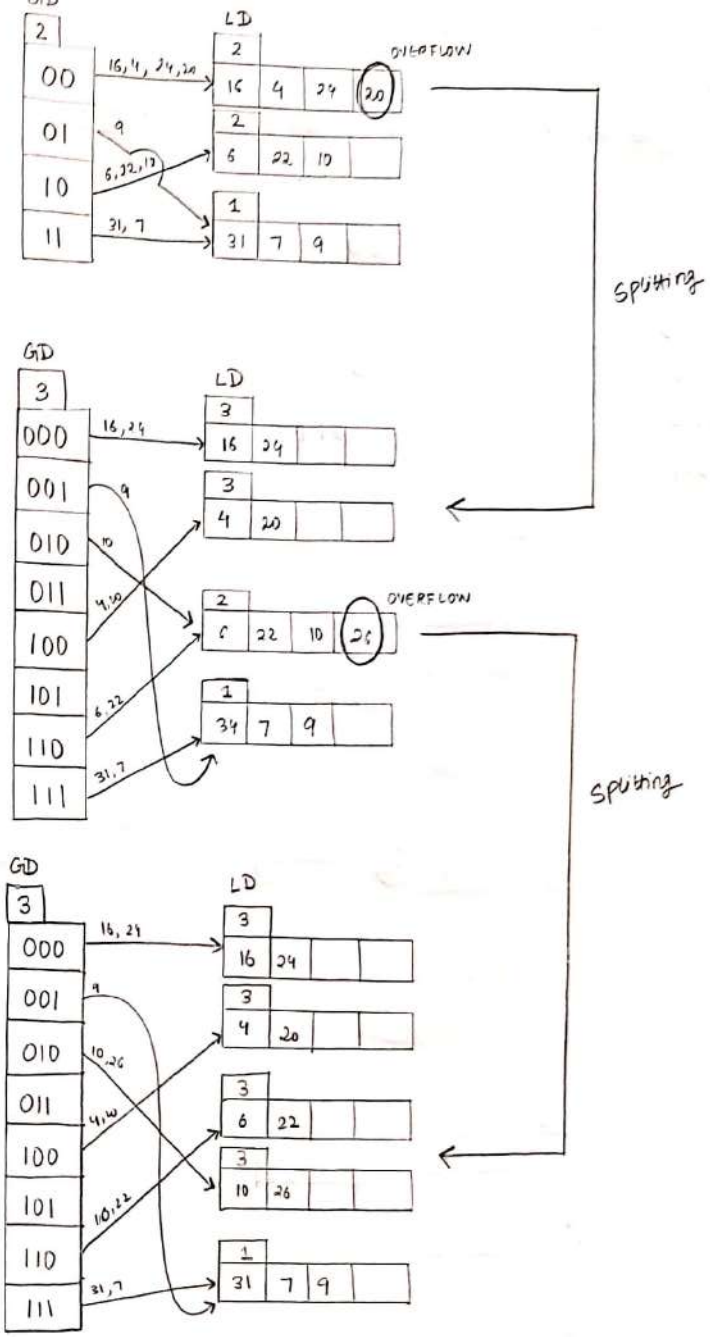
limitations:

1) Size of every bucket is fixed.
2) Memory is wasted in pointers, when local depth and global depth diff becomes drastic.
3) This method is very complicated to code.

16 : 10000
4 : 00100
6 : 00110
22 : 10110
24 : 11000

10 : 01010
31 : 11111
7 : 00111
9 : 01001
20 : 10100
26 : 11010





Q) Keys: 17, 5, 6, 22, 24, 11, 30, 7, 10, 21, 27

Bucket size: 3

Hash f^n: Suppose the GD is X, then the hash f^n returns X MSB

17 : 10001

5 : 00101

6 : 00110

22 : 10110

24 : 11000

11 : 01011

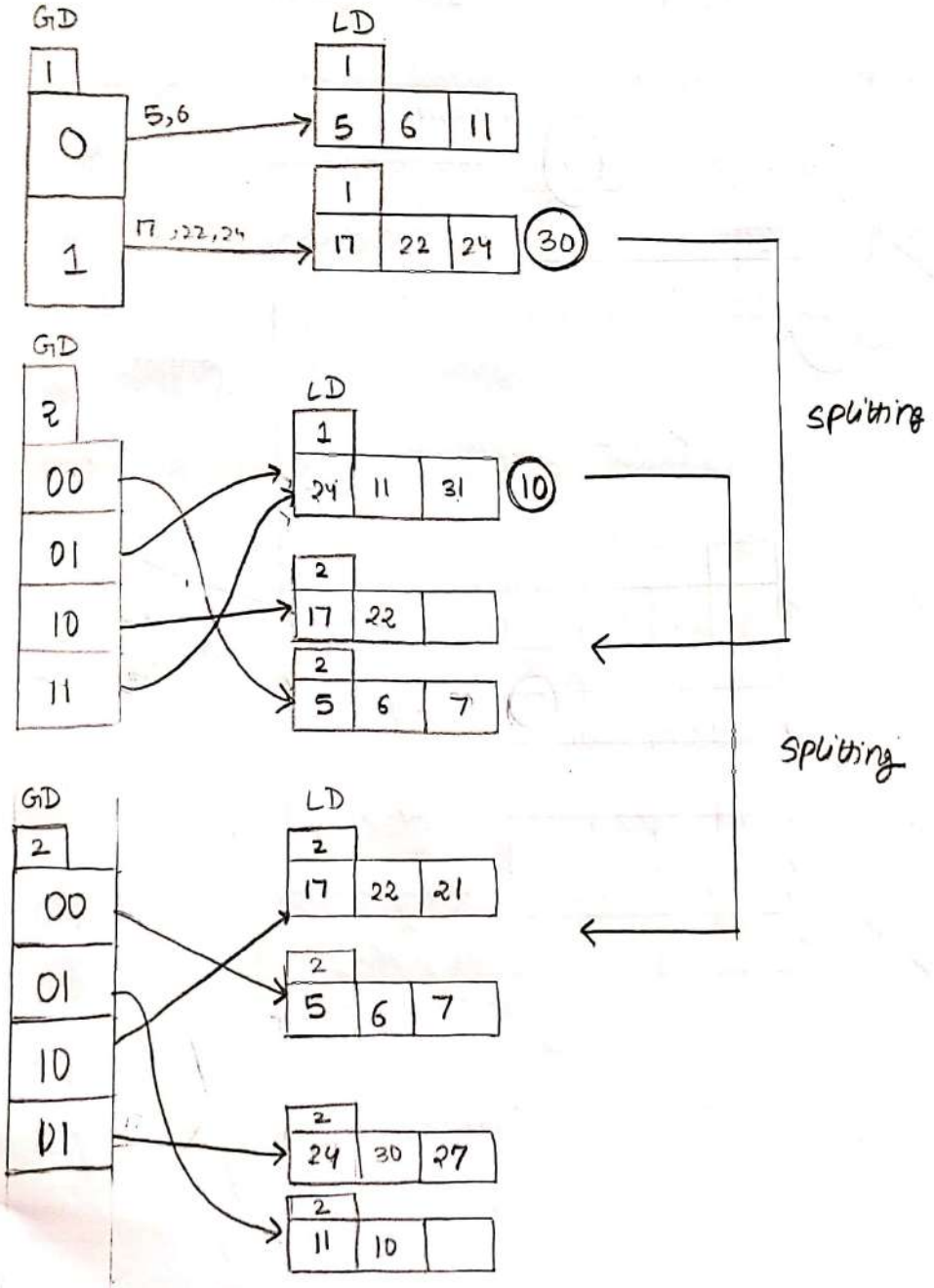
30 : 11110

7 : 00111

10 : 01010

21 : 10101

27 : 11011



Scanned with OKEN Scanner

number of patterns:

1) Double hashing:

$$\frac{1}{\lambda} \log\left(\frac{1}{1-\lambda}\right) \rightarrow \text{successfull search}$$

$$\left(\frac{1}{1-\lambda}\right) \rightarrow \text{unsuccessfull search}$$

2) Quadratic hashing:

$$1 - \log(1-\lambda) - \frac{\lambda}{2} \rightarrow \text{successfull search}$$

$$\frac{1}{1-\lambda} - \lambda - \log(1-\lambda) \rightarrow \text{unsuccessful search}$$

3) Linear hashing:

$$\frac{1}{2}\left(1 + \frac{1}{(1-\lambda)^2}\right) \rightarrow \text{successful search}$$

$$\frac{1}{2}\left(1 + \frac{1}{1-\lambda}\right) \rightarrow \text{unsuccessful search}$$

Scanned with OKEN Scanner

Dictionary

→ A dictionary is a dynamic set ADT.

→ ADT is an object with a generic discription, independent of implementation details.

→ A dictionary is a container of elements, where each element is a pair of key and value and every value is associated with the corresponding keys.

→ It is also a type of DS.

Basic operations of dictionary ADT.

1. Insert (X, D): It insert element (x) in dictionary (D)
2. Delete (X, D): Delete (X) from dictionary (D)
3. Search (X, D): Search element (X) in (D)
4. Member (X, D): Gives true if (X) belong to (D)
5. Size (D): Returns the size of dictionary.
6. Max (D): Returns the maximum/ biggest element of (D)

Ways of implementation.

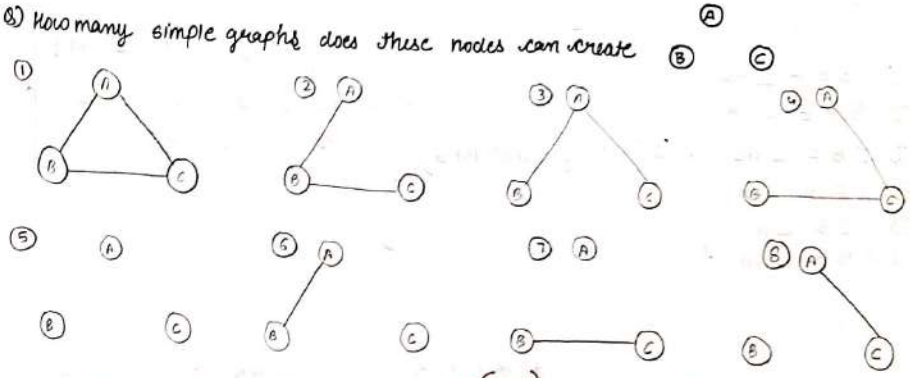
1) Fixed length array
2) Sorted linked list.
3) Hashing
4) Trees (BST, Balanced BST, Splay, M-way, Tries)

Graphs

Simple graph properties:

→ Undirected
→ Unweighted
→ No self loops
→ No parallel edges.

Q) How many simple graphs does these nodes can create



no. of max^m possible edges: $\frac{n(n-1)}{2}$ ∴ n → nodes

no. of simple graphs: 2^n ∴ n → edges

BFS	DFS
→ BREADTH FIRST SEARCH.	→ DEPTH FIRST SEARCH
→ In BFS nodes are traversed level by level.	→ In DFS nodes are traversed depth wise.
→ BFS uses QUEUE data str.	→ DFS uses STACK data str.
→ BFS is slower as memory consumption is high.	→ DFS is faster as it requires less memory.
→ Does not support back tracking.	→ Supports backtracking.
→ It is suitable for finding the shortest path.	→ Suitable for topological sorting & finding the strongly connected componen
→ Time complexity $O(\|V\|+\|E\|)$.	→ Time complexity $O(\|V\|+\|E\|)$.

Scanned with OKEN Scanner